Computación y estructuras discretas II
Grupo 3 Semestre 2026-1

SOLUCIÓN — PreSeguimiento #3 **A**

Nombre: _____ Código: _____

P.1. **(2.5 pts)** Usando el **método del árbol de recursión**, analice y resuelva la siguiente relación de recurrencia:

$$T(n) \;=\; 2\,T\left(\tfrac{n}{4}\right) + 2n + 1, \qquad T(1) = 1$$

Realice el **bosquejo del árbol**, indique la **altura**, el **número de hojas**, la **sumatoria del costo de las hojas** $T(1)$, el **polinomio** resultante y la **complejidad** $\mathcal{O}$.

Solución.

Parámetros: $a = 2, \quad b = 4, \quad f(n) = 2n + 1.$

Expansión del árbol (niveles 0, 1 y 2):
Cada llamado se expande como:

$$T\left(\tfrac{n}{4}\right) = 2\,T\left(\tfrac{n}{16}\right) + 2 \cdot \tfrac{n}{4} + 1 \qquad \text{y} \qquad T\left(\tfrac{n}{16}\right) = 2\,T\left(\tfrac{n}{64}\right) + 2 \cdot \tfrac{n}{16} + 1$$

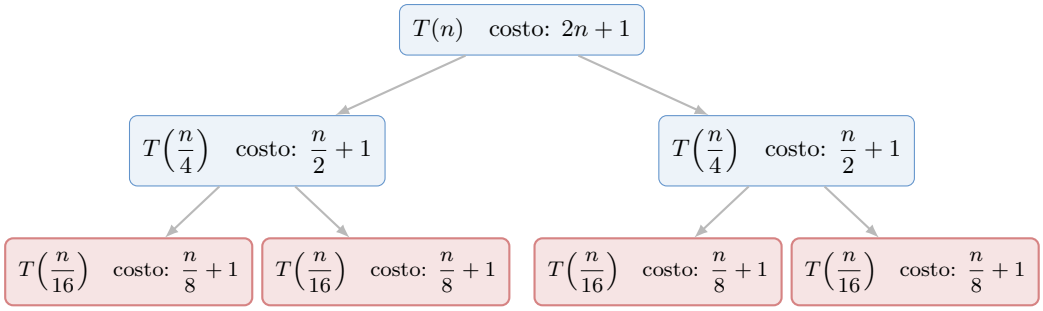

Tabla de análisis:

Nivel i	Nódos	Tamaño subprob.	Costo por nodo	Costo total del nivel
0	1	n	$2n+1$	$2n+1$
1	2	$n/4$	$\frac{n}{2}+1$	$2\cdot\left(\frac{n}{2}+1\right) = n+2$
2	4	$n/16$	$\frac{n}{8}+1$	$4\cdot\left(\frac{n}{8}+1\right) = \frac{n}{2}+4$
i	2^i	$n/4^i$	$2\frac{n}{4^i}+1$	$2^i\cdot\left(2\frac{n}{4^i}+1\right) = 2n\left(\frac{1}{2}\right)^i + 2^i$

Verificación del patrón para $i = 0, 1, 2$:

$$i = 0: \ 2n\left(\tfrac{1}{2}\right)^0 + 2^0 = 2n + 1 \checkmark \qquad i = 1: \ 2n \cdot \tfrac{1}{2} + 2 = n + 2 \checkmark \qquad i = 2: \ 2n \cdot \tfrac{1}{4} + 4 = \tfrac{n}{2} + 4 \checkmark$$

Altura del árbol. El caso base ocurre cuando $n/4^i = 1$, es decir $4^i = n$, luego:

$$h = \log_4 n$$

Número de hojas.

$$2^h = 2^{\log_4 n} = n^{\log_4 2} = n^{1/2} = \sqrt{n}$$

Costo total de las hojas.

$$\sqrt{n} \cdot T(1) = \sqrt{n} \cdot 1 = \sqrt{n}$$

Sumatoria de los nodos internos (niveles $i = 0$ hasta $\log_4 n - 1$):

$$S = \sum_{i=0}^{\log_4 n - 1} \left[2n\left(\tfrac{1}{2}\right)^i + 2^i\right] = 2n \sum_{i=0}^{\log_4 n - 1} \left(\tfrac{1}{2}\right)^i + \sum_{i=0}^{\log_4 n - 1} 2^i$$

Aplicando la fórmula de la serie geométrica $\displaystyle\sum_{k=0}^{n} ar^k = \frac{ar^{n+1} - a}{r - 1}$:

$$\sum_{i=0}^{\log_4 n - 1} \left(\tfrac{1}{2}\right)^i = \frac{\left(\frac{1}{2}\right)^{\log_4 n} - 1}{\frac{1}{2} - 1} = \frac{n^{\log_4(1/2)} - 1}{-\frac{1}{2}} = -2\left(n^{-1/2} - 1\right) = 2\left(1 - n^{-1/2}\right)$$

$$\sum_{i=0}^{\log_4 n - 1} 2^i = \frac{2^{\log_4 n} - 1}{2 - 1} = n^{\log_4 2} - 1 = n^{1/2} - 1 = \sqrt{n} - 1$$

Por lo tanto:

$$\begin{aligned} S &= 2n \cdot 2\left(1 - n^{-1/2}\right) + \sqrt{n} - 1 \\ &= 4n\left(1 - n^{-1/2}\right) + \sqrt{n} - 1 \\ &= 4n - 4n^{1/2} + \sqrt{n} - 1 \\ &= 4n - 3\sqrt{n} - 1 \end{aligned}$$

Polinomio total (nodos internos + hojas):

$$T(n) = S + \sqrt{n} = \left(4n - 3\sqrt{n} - 1\right) + \sqrt{n} = \boxed{4n - 2\sqrt{n} - 1}$$

El término de mayor grado es $4n$, luego la complejidad es:

$$\mathcal{O}(n)$$

a	b	d	Caso del Teorema Maestro	Complejidad
2	4	1	$a < b^d \ \Rightarrow \ 2 < 4^1 = 2 < 4 = true$	$\mathcal{O}(n^d) = \mathcal{O}(n^1) = \mathcal{O}(n)$ ✓

Verificación: Ambos métodos producen $\mathcal{O}(n)$. Coincide con árbol de recursión: **Sí** ✓

P.2. **(2.5 pts)** Usando reconocimiento de patrones en una función recursiva de cola, programe en **Scala** usando listas recursivas una función que reciba como parámetro una lista de enteros. La función debe retornar la **longitud de la racha contigua más larga de números pares** presente en la lista.

```
def maxRachaPares(l: List[Int]): Int
```

No usar variables mutables, ciclos ni `map`/`filter`/`foldLeft`. Usar **recursión de cola** (`@tailrec`) y **pattern matching** con guardas.

| ...

Solución. La función auxiliar interna `aux` recibe tres parámetros: ... ... El acumulador inicial es `rachaActual = 0`, `maxRacha = 0`.

Traza `maxRachaPares(List(2,4,6,1,8,10,3,5))`:

...

```
import scala.annotation.tailrec

def maxRachaPares(l: List[Int]): Int = {
  @tailrec
  def aux(lista: List[Int], rachaActual: Int, maxRacha: Int): Int =
    lista match {
      case Nil => maxRacha
      case x :: xs if x % 2 == 0 =>
        aux(xs, rachaActual + 1, if (rachaActual + 1 > maxRacha) rachaActual + 1 else
            maxRacha)
      case x :: xs =>
        aux(xs, 0, maxRacha)
    }
  aux(l, 0, 0)
}
```

Verificación de los ejemplos:

Ejemplo 1: `List(2, 4, 6, 1, 8, 10, 3, 5)`

Elemento	`x % 2 == 0`	`rachaActual`	`maxRacha`	Acción
2	Sí	1	1	Racha continúa
4	Sí	2	2	Racha continúa
6	Sí	3	3	Racha continúa
1	No	0	3	Racha se reinicia
8	Sí	1	3	Nueva racha
10	Sí	2	3	Racha continúa
3	No	0	3	Racha se reinicia
5	No	0	3	Racha se reinicia
`Nil`		–	**3**	Resultado final

Ejemplo 2: `List(1, 3, 5, 7, 9)` Todos impares ⇒ `rachaActual` y `maxRacha` permanecen en 0 ⇒ resultado: 0 ✓